Retrieval-Augmented Generation

Day 1: Retrieval without an LLM

Retrieval-augmented generation, usually shortened to RAG, combines information retrieval with text generation. This first-day project deliberately builds only the retrieval half. It loads a PDF, divides the extracted text into chunks, embeds each chunk as a vector, and searches those vectors for the passages closest to a question.

An embedding is a numeric representation of meaning. Related passages tend to have nearby vectors even when they do not use exactly the same words. The embedding model does not compose an answer; it only produces numbers that support semantic comparison.

Today's boundary

> The output is source text, not a generated response. Keeping retrieval separate makes it easier to inspect whether the correct evidence was found before an LLM is introduced.

Why split documents into chunks?

A complete document is often too broad for precise retrieval. Splitting creates smaller units so a query can match the paragraph that contains the useful evidence instead of matching an entire PDF. Chunk size therefore controls the balance between context and precision.

Overlap repeats a small amount of text between neighboring chunks. It helps preserve an idea that crosses a chunk boundary. Too little overlap can cut related sentences apart; too much overlap creates duplication and increases indexing work.

Metadata survives splitting

LangChain Documents store text in page_content and descriptive fields in metadata. When the splitter creates chunks, metadata such as the source filename and page number is retained. Retrieval results can therefore point back to their source.

For this small example, character-based chunk sizes are easy to observe. Production systems may use token-aware splitters, document structure, headings, tables, or domain-specific boundaries.

FAISS and Top-K retrieval

FAISS is a library for efficient similarity search over vectors. In this project, LangChain embeds every chunk and inserts the resulting vectors into a FAISS index held in memory. The same embedding model converts the user's question into a query vector.

FAISS compares the query vector with indexed chunk vectors and returns the nearest matches. Top K means the number of results to return. With K set to three, the pipeline prints the three chunks judged most semantically similar to the question.

What the distance means

The example prints a distance beside each result. With the normalized embeddings and FAISS setup used here, a smaller distance indicates a closer match. Distances are useful for comparison within one run, but they are not universal confidence percentages.

The full first-day data flow is: PDF file to page Documents, page Documents to overlapping chunks, chunk text to embedding vectors, vectors to a FAISS index, and a query to the Top 3 source passages.